NATIONAL ECONOMICS UNIVERSITY

MACHINE LEARNING OPERATIONS

PROJECT:

VIETNAMESE NAME RECOGNITION SYSTEM (SPEED-TO-TEXT)

Members: Nguyen Thi Mai Anh
Pham Thi Ngoc Anh
Nguyen Thi Huong Giang
Nguyen Khanh Huyen
Le Lan Huong
Nguyen Thanh Mo

Class: DSEB 65B

Supervisor: Dr. Nguyen Manh Toan

Hanoi, April 2026

1 Introduction

1.1 Problem Statement

Automatic Speech Recognition (ASR) is widely applied in chatbots, voice assistants, and call center automation. However, recognizing Vietnamese personal names remains a challenging task due to several factors.

Vietnamese is a tonal language with six distinct tones, in which subtle variations in pitch can entirely alter the meaning of a word. In real-world call center environments, input quality is further degraded by background noise, inconsistent recording conditions, and regional accent variations across local dialects. Furthermore, names are typically embedded within longer utterances, making extraction considerably more difficult than standard transcription tasks.

Inaccurate name recognition can lead to incorrect customer identification, increased Average Handle Time (AHT), and poor user experience. Therefore, improving name recognition accuracy is important for real-world applications.

Beyond model accuracy, deploying ASR systems in production introduces a distinct set of challenges such as data variability, performance degradation over time, and lack of monitoring. These considerations motivate the need for a structured MLOps pipeline, rather than focusing solely on model-level improvements.

1.2 Objectives

The objective of this project is to design an end-to-end MLOps pipeline for Vietnamese name recognition from speech. The system consists of four main stages:

1. Data preprocessing and versioning.
2. Model training and experiment tracking.
3. Deployment as a REST API.
4. Monitoring to issues and support retraining

   The focus of this project is not only on model accuracy but also on building a reproducible and maintainable system.

1.3 Tool Choices Justification

Tool selection was driven by the specific technical requirements of the problem:

DVC: addresses the challenge of versioning large audio datasets, ensuring reproducibility across experimental runs without overloading the source control system.

MLflow: systematically tracks hyperparameters, evaluation metrics, and model artifacts, ensuring transparency and comparability across training runs.

GitHub Actions: enables automated CI for code quality and testing, preventing regressions from entering the pipeline.

FastAPI : serves the model with low latency, meeting the real-time processing demands of call center environments.

Docker: ensures a consistent runtime environment between development and deployment stages.

Kubernetes: provides the scalability and fault tolerance required for stable production-grade deployment.

Prometheus: enables continuous performance monitoring and early detection of model drift over time.

2 Data Description and Preprocessing

2.1 Dataset Description

The project utilizes two primary datasets, both managed using DVC to ensure reproducibility and efficient version control:

The training dataset (`data_train_70.6`) consists of 2,081 audio samples ($\sim$ 282.6 MB), comprising both original and augmented recordings collected from diverse sources, including video-extracted segments and synthetic speech (TTS). Each audio file is paired with a transcription in a `metadata.csv` file.

To improve model robustness, augmentation techniques such as noise injection, speed and pitch variation, and echo effects are applied to simulate real-world call center conditions.

The evaluation dataset (`person_name_500`) contains 500 recordings ($\sim$ 16.5 MB) of Vietnamese personal names, collected under natural conditions with mild background noise. Despite its practical design, both datasets present limitations, including limited size, heterogeneous sources, and the presence of synthetic speech, which may affect model generalization.

2.2 Preprocessing and Data Quality

Data quality is ensured through a strict preprocessing pipeline implemented in `eval\_wav2vec2.py`. The key normalization steps are summarized in Table 1.

Table 1: Data Preprocessing Steps

Step	**Method**	**Description**
Audio Resampling	`datasets.Audio`	Audio data is resampled to 16 kHz to ensure consistency with the pre-trained Wav2Vec2 model.
Feature Extraction	`Wav2Vec2Processor`	Raw waveforms are converted into model-compatible input representations.
Text Lowercasing	`str.lower()`	All transcripts are converted to lowercase.
Punctuation Removal	`re.sub()`	Special characters and punctuation are removed from transcripts.
Whitespace Normalization	`str.strip()`	Spaces between words are preserved to maintain correct word boundaries.

3 Model Selection and Experiments

3.1 Model Architecture

The team adopted the Wav2Vec 2.0 CTC architecture, which utilizes self-supervised learning on raw audio to build rich feature representations. Specifically, the `nguyenvulebinh/wav2vec2-large-vi-vlsp2020` model was selected as the foundation. Instead of training from scratch, the team employed transfer learning, fine-tuning the network using the Connectionist Temporal Classification (CTC) loss function to map unsegmented audio directly to text.

3.2 Training Setup and Optimization Strategy

The training process was conducted within a Docker container environment (`wav2vec2-train`) to ensure hardware independence and reproducibility. All hyperparameter configurations were systematically tracked and logged via MLflow for experimental transparency.

Several key optimization techniques were employed throughout the training pipeline. The model was trained over 35 epochs with a learning rate of 1×10^{-5}. Gradient Accumulation over 6 steps was applied to simulate an effective batch size of 48, facilitating more stable convergence under GPU memory (VRAM) constraints. Additionally, an 8-bit AdamW optimizer from the `bitsandbytes` library was utilized to reduce memory overhead while preserving model performance.

A notable contribution of this project is the implementation of a custom `CurriculumTrainer`. This module automatically computes the CTC loss after each epoch and applies a Hard-example Mining strategy, performing additional sampling on the 30% of training samples with the highest error rates. This approach compels the model to progressively focus on acoustically challenging syllables and complex regional accent variations, thereby improving robustness on difficult speech patterns.

3.3 Evaluation and Result Analysis

Beyond the standard Word Error Rate (WER) metric, the team developed a custom identity-based evaluation framework grounded in audio filename conventions (e.g., `bui_duc_kha.wav`). This evaluation pipeline applies two distinct matching criteria to assess transcription accuracy at the identity level.

The first criterion, referred to as Strict Matching, requires absolute syllabic precision between the predicted transcription and the ground-truth label, making it sensitive to any deviation in character-level output. The second criterion, referred to as Robust Matching, employs a dedicated `compare_support_dialect_tone` logic to accommodate linguistically acceptable orthographic variations, including vowel interchangeability (e.g., y↔i) and regional tone-marking conventions. This dual-criteria approach enables a more nuanced and linguistically informed assessment of model performance, particularly in the context of Vietnamese dialectal diversity.

3.4 Experimental Results and Analysis

All hyperparameter configurations and training metrics were systematically tracked using MLflow. The final fine-tuned model converged successfully after 35 epochs, achieving a validation loss of 0.520 and a validation WER of 0.223.

To rigorously evaluate the system in a production-like environment, the team conducted an Ablation Study on a specialized test set of 500 Vietnamese personal names. The results are summarized in Table 2.

Table 2: Experimental Results

Step	Run Name	Model Size	Fine-tuned	Preprocessing	PASS Rate (%)	WER	CER
E1	Baseline Base	Base	✗	✗	49.80%	0.792	0.536
E2	Baseline Large	Large	✗	✗	67.40%	0.675	0.478
E3	Finetune Only	Large	✓	✗	**75.00%**	**0.581**	**0.431**
E4	Finetune + Data Eng	Large	✓	✓	70.20%	0.589	0.419

Fine-tuning the model yielded an accuracy gain of 8.2% over the original large pre-trained model. The combination of Curriculum Learning and rigorous audio preprocessing steps — specifically `pydub` for volume normalization and `noisereduce` for background noise filtering — directly contributed to improved recognition accuracy in the noisy call center environments targeted by this system.

The ablation study highlights two key insights. First, transfer learning proved highly effective: upgrading from the Base to the Large pre-trained model (E1 → E2) yielded a 17.6% improvement in PASS rate, and subsequent fine-tuning on the target dataset (E2 → E3) contributed an additional 7.6% gain, bringing overall accuracy to 75.00%. Second, the audio preprocessing pipeline (E4) produced an unexpected performance regression, with the PASS rate declining to 69.20% — contrary to the initial hypothesis that amplitude normalization and noise reduction would be beneficial.

Wav2Vec 2.0's self-supervised architecture is inherently robust to acoustic variations, rendering aggressive algorithmic denoising counterproductive. Experimental results (E4) confirm that such preprocessing distorts critical phonetic and tonal artifacts, leading to performance regression. Consequently, the team recommends bypassing the denoising stage (deploying E3) for production to maximize inference accuracy and minimize computational latency.

4 System Architecture and MLOps Pipeline

4.1 Continuous Integration (CI)

The team implemented a CI pipeline using GitHub Actions to maintain both codebase stability and data processing integrity — a critical distinction from traditional software CI, as undetected failures in data logic can silently corrupt model inference. The pipeline enforces a mandatory Quality Gate on every push and Pull Request, comprising two stages: static code analysis via Ruff and Flake8 to ensure PEP8 compliance and early syntax detection, and automated unit testing via Pytest with a suite of 36 passing test cases. Beyond conventional software validation, these tests function as a Data CI layer, systematically verifying the correctness and determinism of core data transformation routines — including audio preprocessing and Vietnamese text normalization. Successful passage of this gate is enforced as a prerequisite before the Continuous Delivery pipeline proceeds to model packaging and deployment.

4.2 Containerization and Continuous Delivery (CD)

To eliminate environment inconsistency across development and production, the team adopted Docker containerization throughout the ML lifecycle. A dedicated container manages the fine-tuning environment, handling model training with securely injected Hugging Face and W&B credentials. Upon successful passage of all CI Quality Gates, the CD pipeline — orchestrated via GitHub Actions — automatically packages the trained Wav2Vec2 model artifacts and inference logic into an immutable Docker image. The inference service is built on FastAPI, selected for its ASGI asynchronous architecture to minimize inference latency, and the resulting image is published to the GitHub Container Registry (GHCR). To enforce MLOps governance, each image is tagged with its corresponding Git commit SHA, establishing full traceability between the deployed model and its originating codebase and hyperparameter configuration. All sensitive credentials are strictly isolated and injected via GitHub Secrets to maintain deployment security.

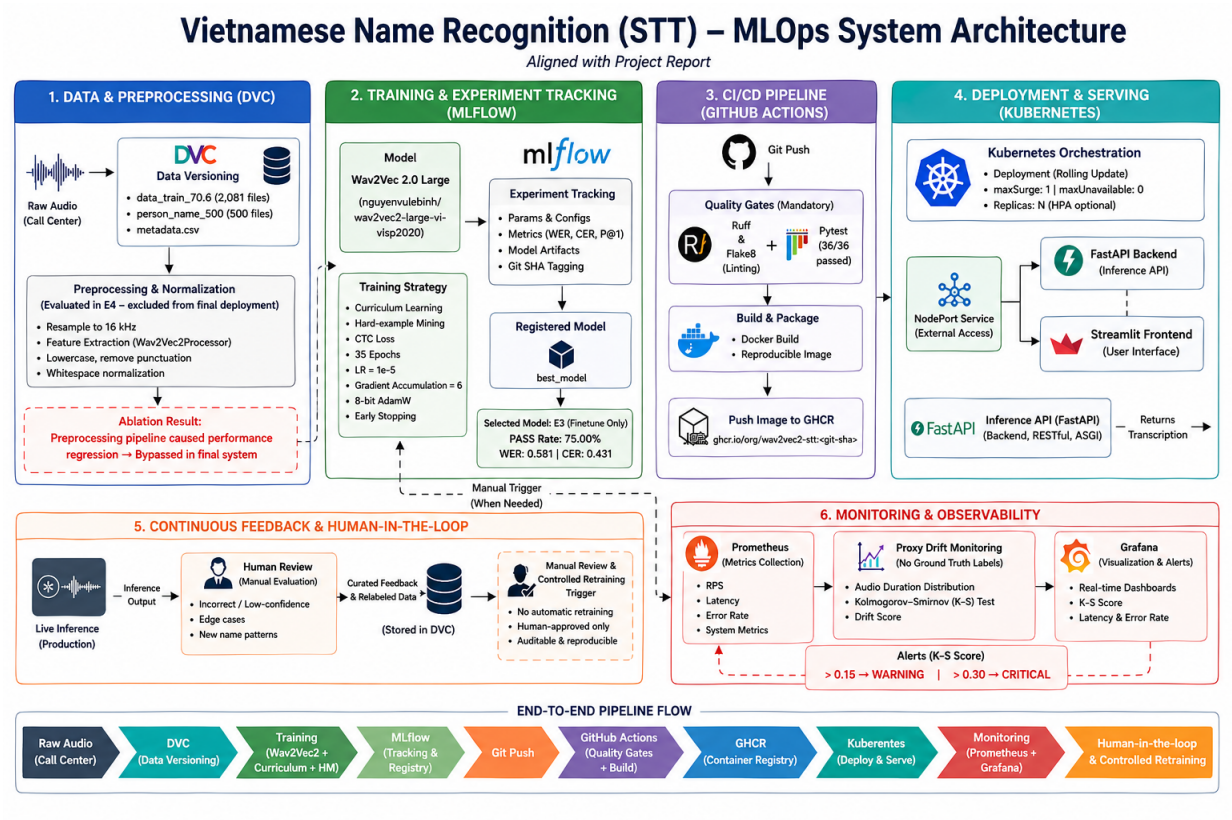
4.3 Kubernetes Deployment

The system operates on a Kubernetes (K8s) cluster to optimize orchestration and scalability. The team applies a Rolling Update strategy with `maxSurge: 1` and `maxUnavailable: 0`. This configuration allows Kubernetes to gradually replace old Pods with newer versions, ensuring the speech recognition service remains available 24/7 with zero downtime. The Deployment resource manages multiple replicas of the API, combined with a NodePort Service to safely expose the communication interface to external clients.

4.4 Monitoring and Observability

Post-deployment system health is monitored through a Prometheus and Grafana stack, where Prometheus continuously scrapes infrastructure metrics — including CPU usage, memory consumption, and API inference latency — and Grafana renders these as real-time interactive dashboards for the MLOps team.

Since ground-truth transcriptions are unavailable in production, direct accuracy monitoring is infeasible. To mitigate silent model degradation, the system instead tracks Data Drift via proxy metrics. Incoming audio requests are processed using `librosa` to extract audio duration distributions, which are then evaluated against the baseline training distribution using the Kolmogorov-Smirnov two-sample test (`ks_2samp`). The resulting K-S statistic is continuously exposed as a Prometheus Gauge metric and visualized across both Grafana and the Streamlit interface under a multi-tier alerting scheme: a score exceeding 0.15 triggers a minor warning, while a score above 0.3 raises a critical drift alert. Rather than relying on automated retraining, these alerts initiate a human-in-the-loop review process, prompting the MLOps team to manually curate feedback data and activate the Continuous Training pipeline — ensuring model updates remain controlled and auditable.



4.5 Web Interface

A web -based interface has been developed to allow users to interact with the system directly. Users can access the application and perform real-time predictions via the following link: Live Web Application

5 Results and Conclusion

5.1 Discussion of Results

This project successfully transitioned a static research model into a production-ready, scalable MLOps system. The final deployed model (Run E3) achieved a PASS rate of 75.00% on a 500-sample Vietnamese name evaluation set, representing a substantial improvement over zero-shot baseline performance. A key finding from the ablation study was the "Preprocessing Paradox," in which aggressive audio normalization and stationary noise reduction were found to degrade phonetic recognition — contrary to conventional ML assumptions. Adopting a data-driven approach by bypassing these preprocessing stages allowed the model to fully leverage its self-supervised feature extraction capabilities. The resulting system operates as a stable REST API on a Kubernetes cluster with a Rolling Update deployment strategy, ensuring zero-downtime inference, low latency, and robust reliability in real-world production environments.

5.2 Limitations and Future Work

Despite the system's positive outcomes, several limitations remain. On the modeling side, the system struggles with extreme regional dialects and single-syllable phonetic ambiguities. On the MLOps side, while Data Drift detection is implemented via K-S testing on input audio duration, real-time Concept Drift monitoring remains infeasible due to the absence of ground-truth labels at inference time.

To address these limitations, two directions are proposed for future work. First, integrating a KenLM language model as a decoder-level component will leverage a predefined Vietnamese name dictionary to resolve phonetic ambiguities and improve transcription accuracy. Second, a human-in-the-loop annotation workflow will be established, wherein samples flagged by the AlertManager for severe drift are routed for manual verification. Upon labeling, these edge-case samples will be versioned via DVC and used to trigger a controlled Continuous Training pipeline, enabling the model to safely adapt to evolving real-world speech patterns.

6 References

References

[1] Nguyen, V. L., et al. *wav2vec2-large-vi-vlsp2020*. Hugging Face Model Hub.

[2] Baevski, A., Zhou, Y., Mohamed, A., and Auli, M. (2020). *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[3] Hugging Face. *Transformers Documentation*. Available at: `https://huggingface.co/docs/transformers`

[4] Kubernetes. *Official Documentation*. Available at: `https://kubernetes.io/docs`

7 Who did what

Team Member	MLOps Role & Technical Responsibilities
Nguyen Thi Mai Anh	**System Architect & MLOps Lead:** Designed the overall system architecture. Developed the core MLOps pipeline including Dockerization, Kubernetes configuration, and recorded the final Video Demo.
Pham Thi Ngoc Anh	**Model Serving & Frontend Engineer:** Developed the Streamlit user interface, integrated the FastAPI endpoints for model inference, and handled real-time audio input processing.
Nguyen Thi Huong Giang	**Data Engineer:** Led the data collection and sourcing process. Implemented data versioning using DVC and developed the audio preprocessing pipeline (`pydub`, `noisereduce`).
Le Lan Huong	**Machine Learning Engineer:** Curated the evaluation dataset (`person_name_500`). Conducted model fine-tuning, executed the ablation study, and tracked experiments systematically via MLflow.
Nguyen Khanh Huyen	**DevOps & QA Engineer:** Collected supplementary audio data. Configured the Continuous Integration (CI) pipeline via GitHub Actions and wrote the comprehensive Pytest suite (36/36 tests) to act as a Data CI gate.
Nguyen Thanh Mo	**Platform & Observability Engineer:** Collected supplementary audio data. Configured the Continuous Delivery (CD) workflow to GHCR and designed the Data Drift monitoring logic (K-S test) using Prometheus.